IET Image Processing

Research Article

# Efficient and unified license plate recognition via lightweight deep neural network

ISSN 1751-9659
Received on 22nd July 2020
Revised 12th October 2020
Accepted on 23rd November 2020
doi: 10.1049/iet-ipr.2020.1130
www.ietdl.org

*Shuxin Qin[1] ✉, Sijiang Liu[2]*

[1]*AInnovation Technology Co., Ltd., No.6 Xingzhi Road, Qixia District, Nanjing, People's Republic of China*
[2]*Nanjing University of Posts and Telecommunications, No.9 Wenyuan Road, Qixia District, Nanjing, People's Republic of China*
✉ *E-mail: qinshuxin@ainnovation.com*

**Abstract:** In this study, the authors are interested in building a unified deep learning framework to solve the recognition problem of both single-line and double-line car license plates. Most existing methods are designed for single-line license plate recognition. For double-line cases, detection and segmentation are usually adopted firstly to locate each line of characters. These methods are usually environmentally sensitive and will bring propagation of error between segmentation and recognition. To solve the problems, the authors propose a unified method that can recognise both single-line and double-line license plates in an end-to-end way without line segmentation and character segmentation. Specifically, an improved lightweight convolutional neural network is used to extract features efficiently. Then, the multi-task learning strategy is used to simultaneously perform license plate classification and character recognition. Finally, recognition task is treated as sequence labelling problems, which are solved by connectionist temporal classification directly. Experimental results indicate that the proposed method significantly outperforms the previous state-of-the-art methods on both public datasets and the synthetic dataset.

## 1 Introduction

In modern life, license plate recognition (LPR) from images as a widely used technology, plays an important role in intelligent transportation system. It is widely used in parking management, security surveillance, traffic control and other fields [1]. LPR algorithms are always required to be accurate and efficient enough to identify each vehicle [2].

Although LPR has been widely used and achieved good results under specific situations, there are still various challenges. Firstly, the large tilt degree of the license plate shooting leads to inaccurate segmentation for both license plate and characters, which directly lower the accuracy of recognition [3]. Secondly, in practical applications, there is a wide variety of license plates. However, most of the existing methods designed for single-line license plates have poor adaptability to double-line cases. Thirdly, running speed has always been an important measure for the recognition algorithms, especially for edge devices applications.

At present, a lot of work has been devoted to solving some of these problems in different ways, and has achieved good results to a certain extent. For single-line cases, the existing works can be simply divided into two broad categories: segmentation-based methods and segmentation-free methods. The first type of method tries to segment and locate each character, and then recognises each character independently by simple classifiers such as support vector machines (SVM) [4], convolutional neural network (CNN) [2, 5] and hybrid classifiers [6]. However, these methods are environmentally sensitive and will bring propagation of error between segmentation and recognition. To overcome these problems, character-level detection can be used to locate more precise character areas [2, 7–9]. Generally, these methods are time consuming and depend on character-level annotations, the manual labelling of which is a heavy workload. The second type of method takes the license plate characters as a sequence directly. The recognition problems can be transformed into sequence labelling problems. To solve this type of problems, CNN-based simple parallel classification methods [10–13] and recurrent neural network (RNN) methods [14–17] are usually adopted to label the license characters directly. The simple parallel classification methods are just designed for fixed number of characters, which limits their application scenarios. Although the RNN methods can achieve better recognition performance, they are time consuming for real-time applications, especially for edge devices and smart cameras applications. For double-line cases, detection and segmentation are usually adopted firstly to locate each line of characters. Then, single-line recognition methods can be used separately for each line. Thus, recognition performance depends heavily on the quality of line segmentation which is sensitive to unconstrained conditions. In fact, there are few methods that can recognise double-line license plates end-to-end without line segmentation.

Summarily, we argue that there are still three main problems that hinder the deployment of LPR systems to real-world applications. Specifically, the first problem is the adaptability to double-line license plates. The second one is the trade-off between speed and accuracy. The convenience of model training and deployment is the third one.

To overcome these problems, we propose an efficient and unified framework to solve the recognition problem for both single-line and double-line car license plates. Specifically, we choose a lightweight deep neural network as the backbone to efficiently extract shared features used for both license plate classification and character recognition. Multi-task learning strategy is adopted for classification and recognition. To be compatible with double-line cases, a classification branch is well designed. Then, recognition branch can compute the final results with the help of classification. We treat characters recognition as sequence labelling without character segmentation. The goal of sequence labelling is to assign sequences of labels [18]. Since there is no correlation between the characters, we use a simple CNN network to simulate temporal classification for efficiency. For training, we only need the sequence-level annotations and the type of license plate, which is easy to obtain compared with character-level annotations. The whole network can be trained end-to-end and work in real time. Fig. 1 describes a schematic overview of our proposed framework.

In summary, the main contributions of this paper are as follows:

(1) We present a novel end-to-end trainable framework to solve LPR task based on sequence labelling method without character segmentation. Compared with segmentation-based methods, end-to-end ones can reduce the propagation of error between segmentation and recognition models.


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

(2) For double-line LPR, we design a multi-task network to automatically discriminate the types of license plate (single-line, double-line) and recognise the license plate characters simultaneously. It is a unified network that is adaptive to single-line and double-line cases.
(3) The proposed method is an efficient and accurate network that can work in real time. A lightweight backbone is used for efficiency. In addition, we only use CNN to simulate temporal classification, which is much more efficient than RNN methods.
(4) The experimental results on different datasets demonstrate that the proposed method significantly outperforms the previous state-of-the-art methods. In addition, through our verification, our model can run well on edge computing devices with lower computational power such as Jetson Nano platform.

The rest of this paper is organised as follows. Section 2 provides a brief discussion on related work. Section 3 describes the details of the proposed method. Section 4 reports the experimental results. Finally, conclusions are drawn in Section 5.

## 2 Related work

Before deep learning technology was widely used, traditional methods for LPR are typically based on characters segmentation. Then, optical character recognition methods are adopted to classify each character one by one. In [1], traditional methods were reviewed and compared from different perspectives. Benefiting from the development of deep learning technology and computing power in recent years, segmentation-free methods have gradually become the prevailing methods of LPR. Here we briefly review the related works including segmentation-based methods, segmentation-free methods and license plate detection methods.

### *2.1 Segmentation-based methods*

Character segmentation is based on the truth that characters and background have obviously different colours in a license plate. Usually, the binary image of a license plate is needed to obtain the boundary of characters by horizontal pixel projection [19–22]. Then, the problem is transformed into single character recognition, which is a kind of image classification task. It should be noted that the accuracy of character recognition depends heavily on the quality of character segmentation.

Template matching methods and learning-based methods are two types of commonly used methods for character recognition. In [20, 23], segmented images are compared with predefined standard template and characters are recognised based on best match. In [22], hybrid discriminative restricted Boltzmann machine is trained for character classification. Traditional machine learning methods, such as hidden Markov models and SVM, can also be used for classification [4, 24]. In recent years, deep learning methods are frequently adopted for this problem. In [8, 25], YOLO framework is used for both character segmentation and recognition. In [26], spatial transform net is used before segmentation and classification to make features corrections. Generally, template matching methods are simple to set up with fewer samples. On the contrary, learning based methods are often more robust by training with a large number of samples. In order to achieve a comprehensive LPR in the real scenes, PixTextGAN is proposed in [27] to generate synthetic license plate images with reasonable text details. Generally, the learning based character recognition methods, combined with character segmentation, are still widely used for LPR in some constrained scenarios.

### *2.2 Segmentation-free methods*

In order to avoid the uncertainty of character segmentation, segmentation-free methods have become a hot research topic for LPR in recent years. Segmentation-free methods have a wider range of applications such as scene text recognition [28] and machine translation, partly due to end-to-end characteristics.

Segmentation-free methods usually take the license plate characters as a sequence directly. The LPR problems can be transformed into sequence labelling problems. To solve this type of problems, both CNN-based and RNN-based methods have been adopted in different situations.

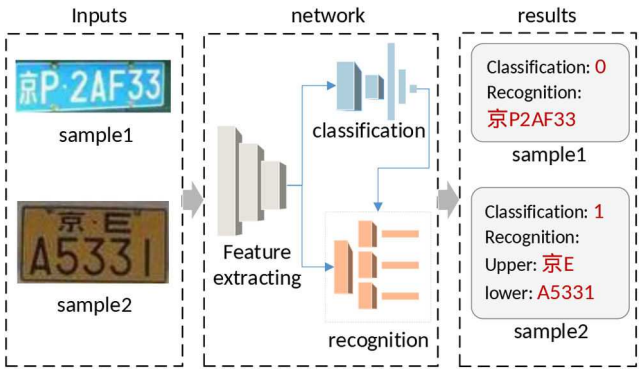


**Fig. 1** *Schematic overview of our proposed framework. Inputs of the present network are different kinds of images from license plate segmentation. Our network consists of feature extracting, classification and recognition. Through end-to-end inference, the network computes outputs including character recognition sequence and classification result. It is a unified method for both single-line and double-line LPR*

In [12], CNN classifier is employed for characters recognition directly. Yang *et al.* [11] used a CNN learns deep features. Then, a kernel-based extreme learning machine classifier is utilised for classification. In [10], multi-layer feature maps are extracted from different layers, resized to a fixed size by ROI Pooling, and then concatenated to one feature map that used for character classification directly. This method is for LPR with fixed length of 7. Similarly, the deep encoder–decoder network proposed by Rakhshani *et al.* [13] extract and transfer features of license plate characters to eight parallel classifiers for recognition. The reason why these methods are feasible is that the length of characters in a license plate is fixed in constrained scenarios. Generally, the adaptability of CNN-based methods is relatively weak, and it is difficult to flexibly adapt to license plates of different lengths using a single model. To solve this problem, character-level detection combined with sequence labelling is proposed in [9]. Similarly, the end-to-end method proposed in [5] detects each character using a regression net, and then recognises each character using a classification net. Other methods such as fully convolutional network (FCN) based methods [2, 7], finding each character area by semantic segmentation, can also achieve relatively good results. Summarily, character detection and segmentation methods depend on character-level annotations, the manual labelling of which is a heavy workload.

RNN-based methods are widely used in speech recognition, language modelling, machine translation and so on. RNN-based methods have natural advantages for sequence labelling due to their powerful connectionist mechanism. In [14], bi-directional recurrent neural network (BRNN) is adopted for solving sequence labelling of license plate characters. Then, the recognition is accomplished by the BRNN and connectionist temporal classification (CTC) [18]. In [15, 16], long short-term memory (LSTM) net is trained to recognise the sequential features extracted from the whole license plate via CNN. In addition, a shared adversarial training network is proposed by Zhang *et al.* [17] to promote the performance of bidirectional LSTM. Generally, RNNs-based methods depend on the computations of the previous time step and therefore do not allow parallel processing over every element in a sequence.

For feature extraction, most of the methods adopt the commonly used backbones, such as Darknet [29], Resnet [30], Densenet [31], which are time consuming. However, lightweight feature extraction net is essential if the computing power is insufficient. For example, the lightweight method in [12] can run in real time on a variety of hardware architectures including low computing power platform. We argue that lightweight networks will be more widely used in real life.


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

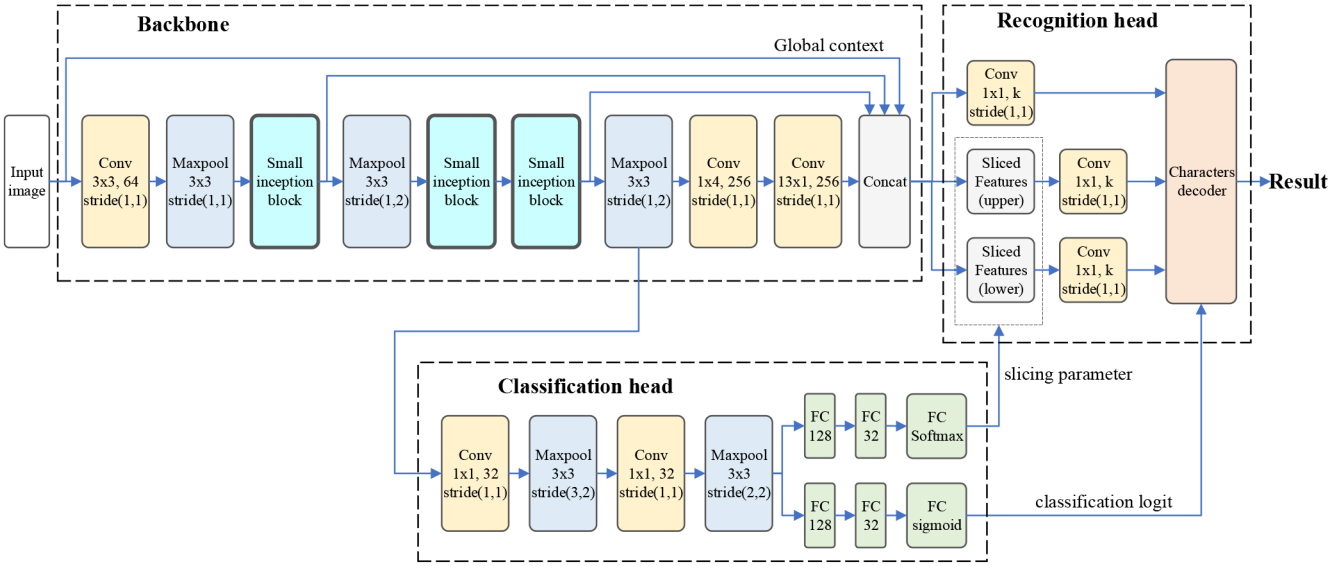


**Fig. 2** *Proposed lightweight network. It consists of three parts: backbone, classification head and recognition head. Backbone is designed to extract features for classification and recognition. Classification head is used to decide the type of license plate and calculate the slicing parameter if the license plate is double-line. With the help of classification, character sequence labelling is completed by the recognition head. It is a lightweight network that can work end-to-end*

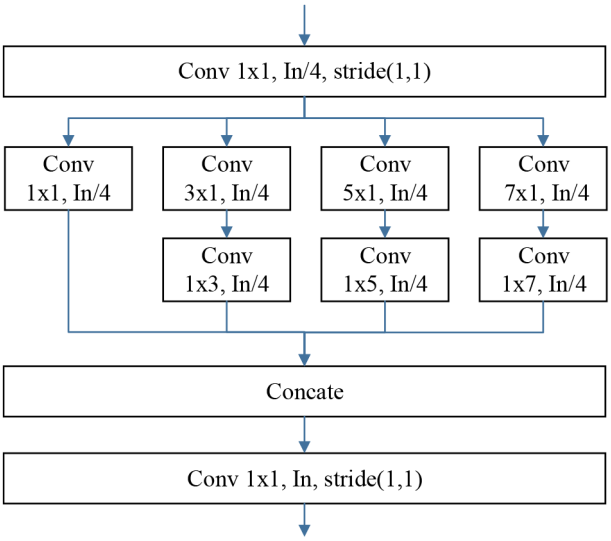


**Fig. 3** *Proposed small inception block. Different sizes of asymmetric convolutions are used to get larger receptive field by adding a small amount of computation*

### 2.3 License plate detection

License plate detection is the premise of LPR. General object detection methods such as YOLO, SSD [32, 33], Faster-rcnn [34], can be used for license plate detection. The bounding box obtained from detection result is used to segment the license plate. However, LPR will produce poor results when the license plates captured at strongly oblique views. To solve this problem, a novel network that can detect and rectify distorted license plates is proposed [3]. For experiments on license plate detection and recognition, we choose the method in [3] to detect the license plates.

## 3 Our method

### 3.1 Overview

Our method is partially based on the network proposed in [12]. We choose the basic recognition pipeline and the lightweight backbone used in [12], and make a number of improvements. In order to enhance the feature extraction ability of the network, a new module called small inception block is designed. In order to support multi-line license plate character recognition, a multi-task learning strategy is used. Specifically, we add a new branch called classification head to determine the license plate types (single-line or double-line) and the slicing parameter. Then, we add two new feature branches to the recognition head. The two featue branches are designed for double-line cases. Finally, the results of classification head are used to choose the proper feature branches for character decoding.

Multi-task learning and end-to-end training are used in our method simultaneously. The license plate classification branch uses a traditional fully connected network for classification, while the LPR branch uses the CTC method for character-level classification directly.

### 3.2 Network design

The architecture of the proposed network is depicted in Fig. 2. It can be divided into three parts: backbone, classification head and recognition head. Specifically, the lightweight backbone is designed to extract features for both classification and recognition. Classification head is used to decide the type of license plate and calculate the slicing parameter if the license plate is double-line. With the help of classification, character sequence labelling is completed by the recognition head. It is a lightweight network that can work end-to-end.

*3.2.1 Backbone:* The lightweight backbone is used to extract image features for classification and recognition. Although some popular architectures are selected frequently, such as Resnet and Densenet, we choose the lightweight net proposed in [12] for efficiency. As we know, a license plate is a rectangle with a specific aspect ratio. Therefore, the input image size ($h \times w$) is set to $36 \times 96$. As shown in Fig. 2, Convolution operation is used to extract features, while MaxPooling operation is adopted to reduce the feature size. Finally, the size of feature maps is reduced to $36 \times 24$. It should be noted that the batch normalisation and ReLU are always adopted after the convolutional layer except for the output layers.

Inspired by inception blocks mentioned in [35], a small inception block is designed to enhance the ability of feature extraction. Compared with the basic block used in [12], our small inception block has larger receptive field by adding a small amount of computation. The small inception block is described in Fig. 3. We use different sizes of asymmetric convolutions that are used to get larger receptive field. It is easy to find that employing asymmetric convolutions, such as a $5 \times 1$ convolution followed by a $1 \times 5$ convolution, can save computational cost dramatically.


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The global context embedding used in [12] is also adopted. We use three branches of global context to enhance the feature maps. Specifically, we use AveragePooling operation to reduce the feature size firstly. Then, the features are normalised by $L_2$ norm. Finally, we concatenate the global contexts to feature maps. In addition, wide convolution (with $13 \times 1$ kernel) used in proposed backbone has been proven to be effective in [12]. Compared with the $3 \times 3$ kernel, there is a small improvement in performance.

*3.2.2 Classification head:* Classification head is composed of convolution, max pooling and full connection layers. There are two branches in classification head. The branch with Softmax activation function is designed to regress the slicing parameter that is used for recognition head to slice features. We adopt discrete classification to simulate the split position parameter, which is used to slice the feature maps of double-line type. It should be noted that this branch is optional. If the height ratio between two lines of characters is almost constant, this branch can be replaced by a fixed parameter. The branch with sigmoid activation function is designed to regress the probability that a license plate is double-line. The output of this branch is used for character decoder.

*3.2.3 Recognition head:* Recognition head is designed to extract final features and decode characters from the final features. There are three branches. The first branch, which extracts the final features of the whole feature maps, works only when the license plate is single-line. The remaining two branches are used to extract the upper and lower features from the whole feature maps, respectively. They work only when the license plate is double-line. The convolution with $1 \times 1$ kernel and $k$ channels is used to extract the final features. $k = C + 1$, where $C$ is the number of character categories. The slicing parameter from classification head is used to slice upper and lower features. The character decoder is designed to decode characters from the final features. The first branch is selected for decoding when $p < 0.5$. Here $p \in (0, 1)$ is the probability value of a license plate being double-line. Otherwise, the remaining two branches are decoded. That is to say, there are three outputs from the recognition head at the same time. Then, the final result is determined by the classification results. When the license plate is single-line, only the first output is used. On the contrary, the second and third outputs are used to decode upper line and lower line characters, respectively. Since we train the network using CTC, the decoding process is to retrieve the most probable labelling $z^*$

$$z^* = \arg\max P(z|x) \tag{1}$$

where $z$ is the labelling path, $x$ is the input sequence to decode. To solve the problem in (1), we choose beam search method, since there is a trade-off between search time and accuracy.

### 3.3 Loss function

The training task includes two parts: classification and recognition. So, the loss function can be formulated as

$$L = L_c + \lambda L_r \tag{2}$$

Here $L_c$ is the classification loss, $L_r$ is the recognition loss. Considering that $L_c$ is easier to converge than $L_r$, we set the weight factor $\lambda = 10$ in our experiments. The classification loss $L_c$ is described in

$$L_c = L_{\text{slice}} + \alpha L_{\text{class}} \tag{3}$$

where $L_{\text{slice}}$ is the Softmax loss function for the branch with Softmax activation function, $L_{\text{class}}$ is the cross-entropy error function for the branch with Sigmoid activation function. The weight factor $\alpha$ is set to 2 in consideration of the convergence speed. $L_{\text{slice}}$ and $L_{\text{class}}$ are defined as follows:

$$L_{\text{slice}} = \frac{1}{N}\sum_{i=0}^{N-1} -\log\left(\frac{e^{p_j}}{\sum_{k=0}^{T-1} e^{p_k}}\right) \tag{4}$$

$$L_{\text{class}} = \frac{1}{N}\sum_{i=0}^{N-1} -[y \times \log(p) + (1-y) \times \log(1-p)] \tag{5}$$

In (4), $N$ is the size of a mini-batch for training, $T$ is the number of categories. Considering that the height of final features is 36, we set $T = 18$ in our experiments to regress a parameter that can divide the height evenly. In (5), $y$ is the target label and $p$ is predicted logit.

The recognition loss $L_r$ also consists of two parts: single-line loss and double-line loss. It can be formulated as

$$L_r = (1-y) \times L_{ctc} + y \times (L_{ctc}^{\text{upper}} + L_{ctc}^{\text{lower}}) \tag{6}$$

where $y$ is the target label of a license plate, $L_{ctc}$ is the single-line CTC loss, $L_{ctc}^{\text{upper}}$ and $L_{ctc}^{\text{lower}}$ are CTC losses for upper and lower characters, respectively. Here the standard CTC loss is adopted. It is given by

$$L_{\text{CTC}} = -\sum_{(x,z) \in S} \ln P(z|x) \tag{7}$$

where $P(z|x)$ represents the conditional probability of the output target sequence $z$ through the input $x$. $S$ is defined as the training dataset. To calculate the CTC loss, the length of time needs to be greater than twice the length of characters. Considering that the number of characters in most license plates is from 6 to 9, we set the length of the final feature for decoding to $L = 24$. Since we use the CNN output to decode directly, the length $L$ is used to indicate the length of time. Defining $p_{\pi_t}^t$ as predict probability of sequence $\pi_t$ at time $t$, $P(z|x)$ can be formulated as

$$P(z|x) = \prod_{t=1}^{L} p_{\pi_t}^t \tag{8}$$

### 3.4 Post-processing

As the license plate is a special character sequence with certain specifications, we can filter the decoding sequences by setting specific rules. That is to say, after CTC search, the results that do not satisfy the rules are discarded firstly.

We formulate corresponding rules for single-line and double-line license plates, including: number of characters rule and character subset rule. The first rule is adopted to filter out the result sequences whose length do not meet the requirements. The second rule defined a valid subset for each character of the license plate. The result sequences will be filtered out if any character is not in its subset. We do post-processing after the beam search. The first sequence that meets the predefined rules is the final result.

## 4 Experiments

In this section, we evaluate our approach on both public datasets (CCPD [10], AOLP [21]) and our synthetic dataset. For performance metrics, the recognition accuracy of the whole license plate is adopted. Experimental comparisons with other state-of-the-art methods are reported.

### 4.1 Datasets

*4.1.1 CCPD dataset:* CCPD is a large-scale and diverse license plate (LP) dataset. It provides over 250k unique LP images with detailed annotations including LP number, LP bounding box, four vertices locations and other information. Specifically, it is divided into nine sub-datasets including Base (common photos with license plates), DB (illuminations are dark, uneven or extremely bright), FN (the distance from LP to shooting location is relatively far or


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

near), Rotate (images taken by rotated cameras), Tilt (images taken by rotated cameras), Blur (blurry due to hand jitter), Weather (images with rain, snow or fog), Challenge (the most challenging images) and NP (images without a LP) [10]. We use the sub-datasets for our experiments except Blur and NP, since they are not designed for recognition. We extract the LP areas using the four vertices and transform them to a fixed size rectangular images for LPR.

*4.1.2 AOLP dataset:* AOLP (application-oriented license plate dataset) consists of 2049 images of Taiwan license plates. It is split into three categories: AC (access control, 681 images), LE (law enforcement, 757 images), and RP (road patrol, 611 images). Specifically, AC refers to the cases that a vehicle passes a fixed passage at a reduced speed or with a full stop, LE refers to the cases that a vehicle is captured by a roadside camera, RP refers to the cases that a vehicle are captured by another moving vehicle [21]. Since there are no annotations for LP vertices locations, we simply borrow the annotated data used in [3] for experiments.

*4.1.3 Synthetic dataset:* As there is no LP dataset for multi-line type, we use the synthetic dataset to validate the present method for double-line recognition. We generate a synthetic Chinese LP dataset including various types of LP (double-line with seven characters, single-line with seven characters and single-line with eight characters). The dataset generation steps are as follows:

(1) Generate the background image using well-prepared LP templates.
(2) Draw LP characters on the generated background image according to the predefined rules.
(3) Do data augmentation, including affine transformation, perspective transformation, illumination and saturation transformation, Gaussian fuzzy and random noise.

Example images of synthetic dataset are shown in Fig. 4. We generate three kinds of Chinese license plates with different lengths, colours and types. As we know, Chinese characters are more complex than letters and numbers, which makes the LPR more challenging.

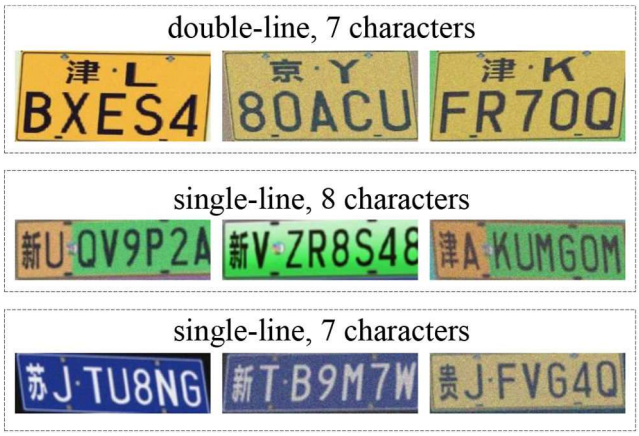


**Fig. 4** *Example images in synthetic dataset*

### *4.2 Implementation details*

Our network is trained with $36 \times 96$ images using the stochastic gradient descent on a GeForce RTX 2080Ti GPU. Learning rate is set to $1 \times 10^{-3}$ initially, and decayed using exponential strategy. The training batch size is set to 128. To prevent over-fitting, we use the data augmentation strategies including colour jittering, noising, blurring and slightly shifting.

Since the present method is mainly to solve the problem of LPR, for license plate detection task, the detection method in [3] is re-implemented as a supplement. A complete solution consisting of license plate detection and the proposed method is also tested.

### *4.3 Experiments on CCPD dataset*

Since the dataset does not provide data splitting for training and test, we randomly divide each subset used for our experiments into two equal parts, just as the baseline method. First, the license plates are segmented and transformed according the four vertices of each annotation. Then, we train the model on the CCPD-Base sub-dataset which has 100 K training samples. For other sub-datasets (DB, FN, Rotate, Tilt, Weather, Challenge), the model is fine-tuned on each training part.

The LPR performance of the present method on CCPD dataset is shown in Table 1. We provide comparison tests and the ablation study on different sub-datasets. Ours (without classification) means that the classification head is removed from our framework for this experiment. Ours (with classification) is the proposed framework with the classification branch. Ours (add post-processing) indicates that the post-processing module is added in the proposed framework. Compared with [12], the average accuracy is improved from 96.35 to 97.34% by using our proposed inception block. Our method (with classification) is the complete network described in Fig. 2. In this experiment, extra synthetic samples are added in training set. Compared with our method (without classification), there is little difference in the results. That is to say, the proposed method is a unified framework that can be compatible with double-line license plates without reducing the original performance for single-line cases.

In addition, The lower accuracy on FN, DB and Challenge sub-datasets are caused by the low lower quality images. Some hard samples from CCPD are shown in Fig. 5. The left image is from DB sub-dataset. Illuminations on its LP area are extremely bright. The middle image from FN sub-dataset is relatively far from the shooting location. The right image from the challenge sub-dataset is collected with a large tilt degree. These samples are hard to recognise even by human. However, our method can also achieve better performance on these sub-datasets compared with [12].

By adding post-processing, the average accuracy is also improved to some extent. Specifically, the difficult sub-datasets, such as DB and Challenge, benefit more from the post-processing. Character features are more likely to be confused in the difficult sub-datasets. Therefore, the results after CTC search are more confused. However, the correct sequence may be in the results but with lower probability. That is to say, more wrong results can be corrected by post-processing. It should be noted that the results for Lprnet are based on the re-implemented version, since there is neither official baseline on this dataset nor source code for test.

**Table 1** Recognition accuracy on different sub-datasets. We provide comparison tests and the ablation study to prove the effectiveness of this approach. The experimental results demonstrate that our network performs better than [12]. In addition, the classification branch designed for double-line recognition has little negative effect on single-line. The post-processing can improve the accuracy more or less

| Method | Base, % | Rotate, % | Tilt, % | FN, % | DB, % | Weather, % | Challenge, % | Avg, % |
|---|---|---|---|---|---|---|---|---|
| Lprnet (2018) [12] | 99.11 | 96.64 | 94.23 | 90.84 | 87.46 | 98.02 | 88.64 | 96.35 |
| Ours (without classification) | 99.84 | 97.64 | 95.89 | 92.11 | 89.44 | 98.83 | 90.09 | 97.34 |
| Ours (with classification) | 99.83 | 97.76 | 95.62 | 92.07 | 89.25 | 98.89 | 90.10 | 97.31 |
| Ours (add post-processing) | 99.86 | 97.81 | 95.87 | 92.69 | 89.92 | 99.18 | 90.87 | 97.51 |


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

### 4.4 Experiments on AOLP dataset

For AOLP dataset, our method is compared with several typical and state-of-the-art methods, including Hsu [21], FCN [7], LSTM [15] and Segmentation [2]. [21] uses traditional character-level detection method to locate each character and uses simple classifier for recognition. [2, 7] use segmentation method to locate each character and CNN method for recognition. The method in [15] is a segmentation-free method based on LSTM. The performances of these methods are presented in Table 2. Ours (with bounding box) means that we use the bounding box to locate and segment the license plates, use the proposed method for recognition. Ours (with four vertices) indicates that we use the four vertices to locate, segment and rectify the license plates, use the proposed method for recognition. Experimental results demonstrate that, our method with bounding box outperforms the LSTM-based method in [15], but is not as good as the semantic segmentation-based method in [2]. However, when we use the four vertices to segment and rectify the license plates, we get the comparable results with the semantic segmentation method. Specifically, ours (with four vertices) achieves higher recognition accuracy on RP sub-dataset and up to 99.26% for average accuracy. But our method is much more efficient than [2], because Resnet101 used in [2] is much more time consuming than our lightweight network.

It should be noted that, the methods with better performance, including FCN [7], Segmentation [2] and Ours (with four vertices), adopt more supervised information for model training. Specifically, The character-wise boxes are used for training in [2], and four vertexes is adopted by us. Accordingly, we argue that more supervised information for deep learning can bring better performance. Generally, our method is more competitive due to the lightweight framework.

**Fig. 5** *Hard samples from CCPD. The left image is from DB sub-dataset. Illuminations on its LP area are extremely bright. The middle image from FN sub-dataset is relatively far from the shooting location. The right image from the challenge sub-dataset is collected with a large tilt degree. These samples are hard to recognise even by human*

**Table 2** Recognition accuracy on AOLP dataset. Ours (with bounding box) means that the license plates are segmented by bounding boxes. Ours (with four vertices) means we used the four vertexes for segmentation. Our method achieves higher recognition accuracy on RP sub-dataset and up to 99.26% for average

| Method | AC, % | LE, % | RP, % | Avg, % |
|---|---|---|---|---|
| Hsu (2013) [21] | 88.50 | 86.60 | 85.70 | 86.93 |
| FCN (2017) [7] | 97.90 | 97.60 | 98.20 | 97.90 |
| LSTM (2018) [15] | 94.85 | 94.19 | 88.38 | 92.47 |
| Segmentation (2018) [2] | **99.41** | **99.31** | 99.02 | 99.25 |
| Ours (with bounding box) | 98.48 | 98.06 | 97.13 | 97.89 |
| Ours (with four vertices) | 99.34 | 99.27 | **99.18** | **99.26** |

Bold values indicate the best results.

**Table 3** Recognition accuracy on Synthetic dataset. We train a single model for different types of license plates recognition. Results demonstrate that the proposed method is adaptive and unified

| Our method | Single-line for 7, % | Single-line for 8, % | Double-line for 7, % | Avg, % |
|---|---|---|---|---|
| Without post-processing | 99.92 | 99.74 | 99.70 | 99.78 |
| With post-processing | **99.95** | **99.80** | **99.75** | **99.83** |

Bold values indicate the best results.

### 4.5 Experiments on synthetic dataset

In order to illustrate the effectiveness for double-line license plates, we choose the synthetic dataset since there is no public dataset for double-line license plates. We train our model on the mixed dataset including the three kinds of license plates: double-line (seven characters), single-line (seven characters) and single-line (eight characters). Each kind has 10$k$ samples for training and 2$k$ for test. The performance is shown in Table 3. Our method achieves 99.78% without post-processing and 99.83% with post-processing for average accuracy. For different types of license plates, our method always works well with minor difference (99.92 and 99.95% for single-line seven characters, 99.74 and 99.80% for single-line eight characters, 99.78 and 99.83% for double-line seven characters). It should be noted that, since the input size is set to $36 \times 96$ for all three kinds of license plates, the lower performance for single-line (eight characters) and double-line samples may be caused by the smaller output feature maps averaging to each character. We argue that, our approach is a unified framework with the ability to deal with different types of license plates by single model.

### 4.6 Experiments for detection and recognition

License plate detection is always needed when the LPR method is used in practical application scene. Experiments above have proved the effectiveness of our method for recognition. However, that is based on the correct segmentation of the license plates. We use the method in [3] for detection and the proposed method for recognition to form a complete solution. It should be noted that the detection method we adopted is re-implemented and trained for license plate segmentation. CCPD dataset and ALOP dataset are used for experiments. Specifically, we choose three representative subsets: CCPD-Base, CCPD-Challenge and ALOP-AP. Comparisons with state-of-the-art methods are shown in Table 4. We compare our solution with state-of-art methods including two representative end-to-end methods [10, 16] and one two-stage (detection and recognition) method [3]. Specifically, [10] provides an end-to-end method with classifier for recognition directly, [16] uses the LSTM method for recognition, and [3] adopts segmentation-based method for recognition.

**Table 4** Performances for detection and recognition. We compare our solution with state-of-art methods including end-to-end methods [10, 16] and two-stage method [3]. Experimental results demonstrate that our solution achieves the highest accuracy

| Method | CCPD-Base, % | CCPD-Challenge, % | ALOP-AP, % |
|---|---|---|---|
| Li (2019) [16] | 97.8 | 81.2 | 88.38 |
| RPnet (2018) [10] | 98.5 | 85.1 | – |
| Sergio (2018) [3] | – | – | 98.36 |
| Det. [3] + Ours | **99.1** | **87.6** | **98.84** |

Bold values indicate the best results.


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

**Fig. 6** *Detection and recognition results on CCPD dataset with our solution. The license plate area is annotated by a quadrilateral which is more accurate than the rectangle. Recognition benefits from the accurate detection. It is easy to find that our method is able to recognise the license plate with a large tilt degree*

Generally, our solution performs better than the other methods. Qualitatively, our solution achieves a recognition accuracy of 99.1% on CCPD-Base sub-dataset, which is 1.3% higher than [16] and 0.6% higher than [10]. For CCPD-Challenge sub-dataset, it is 6.4 and 2.5% higher separately. Compared with the solution in [3], we use the same license plate detection method. For recognition, Silva and Jung [3] adopted the segmentation-based method. For ALOP-AP sub-dataset, our solution also has better performance. Summary, we provide a complete solution for license plate detection and recognition that outperforms the state-of-art methods. Detection and recognition results on CCPD dataset for our solution are shown in Fig. 6.

### *4.7 Speed analysis*

We specially analyse the time consumption of the proposed method on different devices, including CPU (Intel Core(TM) i7 7700), GPU (Nvidia RTX 2080Ti) and NVIDIA Jetson Nano (a low-power embedded platform for edge computing). The experiments are based on the CCPD dataset. As shown in Table 5, our method can run in real-time even on the edge computing device. Jetson Nano with TF means that the Tensorflow version is adopted for testing. We also test the Tensor-RT version of our method named as Jetson Nano with TRT to speed up the computation. The complete solution (Det. [3] + Ours) is also tested to give a clear demonstration of the total time consumption by adding license plate detection. It should be noted that, for all speed experiments, images with size $36 \times 96$ are used for LPR, and images with size $720 \times 1160$ are used for detection.

The experimental results for LPR method in [2] and complete solution in [16] are from their papers. Both of them are based on NVIDIA Titan X GPUs with 12GB memory, which are better than ours. That is to say our method is more than five times faster than [2], which adopts semantic segmentation method with large backbone (Resnet101). Meanwhile, our complete solution is also more than nine times faster than [16]. In addition, our solution can easily run on the Jetson Nano platform at a speed of nearly 8 fps. As we know, Jetson Nano is an entry-level computing platform with just 472 GFlops (FP16) computational power. Most of the smart cameras and edge devices are more powerful. Accordingly, we argue that our solution is more suitable for smart cameras and edge devices.

**Table 5** Time consumption of the proposed method. Our method and solution are significantly faster than other methods. Experiments on Jetson Nano platform demonstrate that our solution can even run on edge devices and smart cameras with low computing power

| Methods | CPU, ms | GPU, ms | Jetson Nano with TF, ms | Jetson Nano with TRT, ms |
|---|---|---|---|---|
| Segmentation (2018) [2] | – | 40 | – | – |
| Ours | 24 | 8 | 68 | 28 |
| Li (2019) [16] | – | 300 | – | – |
| Det. [3] + Ours | 124 | 30 | 674 | 127 |

## 5 Conclusion

In this work, we have presented an efficiency and unified lightweight network for both single-line and double-line LPR. It is an end-to-end method that can automatically read different kinds of license plates without extra processing. Specifically, we have designed a classification head and a multi-branch recognition head to process license plates with different types and lengths. By applying multi-task learning and inferring strategy, the classification head can output slicing parameter and classification result, which are used for character recognition. Character recognition is solved by sequence labelling without using RNN method for efficiency. Comprehensive experiments on popular datasets for license plate detection and recognition clearly demonstrate the advantage of the proposed method. In addition, it can perform in real time even on the low-power embedded devices.

For future work, we plan to integrate license plate detection and recognition into a single network, which should be efficient enough for edge devices. Moreover, we intend to improve the license plate classification branch by learning more attributes, such as colour, which is also useful in practical application scenarios.

## 6 References

[1] Du, S., Ibrahim, M., Shehata, M., *et al.*: 'Automatic license plate recognition (alpr): a state-of-the-art review', *IEEE Trans. Circuits Syst. Video Technol.*, 2013, **23**, (2), pp. 311–325

[2] Zhuang, J., Hou, S., Wang, Z., *et al.*: 'Towards human-level license plate recognition'. Proc. Eur. Conf. Comput. Vis. (ECCV), Munich, Germany, 2018


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[3] Silva, S.M., Jung, C.R.: 'License plate detection and recognition in unconstrained scenarios'. Proc. Eur. Conf. Comput. Vis. (ECCV), Munich, Germany, 2018
[4] Khan, M.A., Sharif, M., Javed, M.Y., *et al.*: 'License number plate recognition system using entropy-based features selection approach with svm', *IET Image Proc.*, 2018, **12**, (2), pp. 200–209
[5] Wang, D., Tian, Y., Geng, W., *et al.*: 'Lpr-net: recognizing chinese license plate in complex environments', *Pattern Recognit. Lett.*, 2020, **130**, pp. 148–156
[6] Ashtari, A.H., Nordin, M.J., Fathy, M.: 'An iranian license plate recognition system based on color features', *IEEE Trans. Intell. Transp. Syst.*, 2014, **15**, (4), pp. 1690–1705
[7] Yue, W., Jianmin, L.: 'License plate recognition using deep fcn'. Proc. Int. Conf. Cognitive Syst. Signal Process. (ICCSIP), Beijing, People's Republic of China, 2016
[8] Hendry, , Chen, R.C.: 'Automatic license plate recognition via sliding-window darknet-yolo deep learning', *Image Vision Comput.*, 2019, **87**, pp. 47–56
[9] Gao, F., Cai, Y., Ge, Y., *et al.*: 'Edf-lpr: a new encoder-decoder framework for license plate recognition', *IET Intell. Trans. Syst.*, 2020, **14**, (8), pp. 959–969
[10] Xu, Z., Yang, W., Meng, A., *et al.*: 'Towards end-to-end license plate detection and recognition: a large dataset and baseline'. Proc. Eur. Conf. Comput. Vis. (ECCV), Munich, Germany, 2018
[11] Yang, Y., Li, D., Duan, Z.: 'Chinese vehicle license plate recognition using kernel-based extreme learning machine with deep convolutional features', *IET Intell. Trans. Syst.*, 2018, **12**, (3), pp. 213–219
[12] Zherzdev, S., Gruzdev, A.: 'Lprnet: License plate recognition via deep neural networks', arXiv:180610447, 2018
[13] Rakhshani, S., Rashedi, E., Nezamabadi-pour, H.: 'Representation learning in a deep network for license plate recognition', *Multimed Tools Appl.*, 2020, **79**, pp. 13267–13289
[14] Wang, J., Huang, H., Qian, X., *et al.*: 'Sequence recognition of chinese license plates', *Neurocomputing*, 2018, **317**, pp. 149–158
[15] Li, H., Wang, P., You, M., *et al.*: 'Reading car license plates using deep neural networks', *Image Vision Comput.*, 2018, **72**, pp. 14–23
[16] Li, H., Wang, P., Shen, C.: 'Toward end-to-end car license plate detection and recognition with deep neural networks', *IEEE Trans. Intell. Transp. Syst.*, 2019, **20**, (3), pp. 1126–1136
[17] Zhang, S., Tang, G., Liu, Y., *et al.*: 'Robust license plate recognition with shared adversarial training network', *IEEE Access*, 2020, **8**, pp. 697–705
[18] Alex, G., Santiago, F., Faustino, G., *et al.*: 'Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks'. Proc. Int. Conf. Mach. Learn. (ICML), Pittsburgh, PA, USA, 2006
[19] Wu, H.-H.P., Chen, H.-H., Wu, R.-J., *et al.*: 'License plate extraction in low resolution video'. Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR), Hong Kong, People's Republic of China, 2006, vol. 1, pp. 824–827
[20] Goel, S., Dabas, S.: 'Vehicle registration plate recognition system using template matching'. Proc. Int. Conf. Signal Process. Commun. (ICSC), Noida, India, 2013, pp. 315–318
[21] Hsu, G., Chen, J., Chung, Y.: 'Application-oriented license plate recognition', *IEEE Trans. Veh. Technol.*, 2013, **62**, (2), pp. 552–561
[22] Gou, C., Wang, K., Yao, Y., *et al.*: 'Vehicle license plate recognition based on extremal regions and restricted boltzmann machines', *IEEE Trans. Intell. Transp. Syst.*, 2016, **17**, (4), pp. 1096–1107
[23] Liu, Y., Huang, H., Cao, J., *et al.*: 'Convolutional neural networks-based intelligent recognition of chinese license plates', *Soft Comput.*, 2018, **22**, pp. 2403–2419
[24] Bulan, O., Kozitsky, V., Ramesh, P., *et al.*: 'Segmentation- and annotation-free license plate recognition with deep localization and failure identification', *IEEE Trans. Intell. Transp. Syst.*, 2017, **18**, (9), pp. 2351–2363
[25] Laroca, R., Severo, E., Zanlorensi, L.A., *et al.*: 'A robust real-time automatic license plate recognition based on the yolo detector'. Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN), Rio de Janeiro, Brazil, 2018, pp. 1–10
[26] Dong, M., liang, H., Luo, C., *et al.*: 'A cnn-based approach for automatic license plate recognition in the wild'. Proc. Conf. British Machine Vision, London, UK, 2017
[27] Wu, S., Zhai, W., Cao, Y.: 'Pixtextgan: structure aware text image synthesis for license plate recognition', *IET Image Process.*, 2019, **13**, (14), pp. 2744–2752
[28] Shi, B., Bai, X., Yao, C.: 'An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition', *IEEE Trans. Pattern Anal. Mach. Intell.*, 2017, **39**, (11), pp. 2298–2304
[29] Redmon, J., Divvala, S., Girshick, R., *et al.*: 'You only look once: unified, real-time object detection'. Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR), Las Vegas, NV, USA, 2016, pp. 779–788
[30] He, K., Zhang, X., Ren, S., *et al.*: 'Deep residual learning for image recognition'. Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR), Las Vegas, NV, USA, 2016
[31] Huang, G., Liu, Z., van der Maaten, L., *et al.*: 'Densely connected convolutional networks'. Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR), Honolulu, HI, USA, 2017
[32] Wei, L., Dragomir, A., Dumitru, E., *et al.*: 'Ssd: single shot multibox detector'. Proc. Eur. Conf. Comput. Vis. (ECCV), Amsterdam, The Netherlands, 2016
[33] Redmon, J., Farhadi, A.: 'Yolov3: An incremental improvement', arXiv:180402767, 2018
[34] Ren, S., He, K., Girshick, R., *et al.*: 'Faster r-cnn: towards real-time object detection with region proposal networks', *IEEE Trans. Pattern Anal. Mach. Intell.*, 2017, **39**, (6), pp. 1137–1149
[35] Szegedy, C., Vanhoucke, V., Ioffe, S., *et al.*: 'Rethinking the inception architecture for computer vision'. Proc. IEEE Conf. Comput. Vis. Pattern Recog. (CVPR), Las Vegas, NV, USA, 2016, pp. 2818–2826


© The Institution of Engineering and Technology 2020

17519667, 2020, 16, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/iet-ipr.2020.1130, Wiley Online Library on [19/12/2022]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License